

Peptacular: A Python package for amino acid sequence analysis with ProForma 2.1

Patrick T. Garrett¹ and John R. Yates III¹

¹ The Scripps Research Institute, United States

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Mark A. Jensen](#)

Reviewers:

- [@gchure](#)

Submitted: 16 March 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Mass spectrometry identifies and characterizes proteins by measuring molecules and their fragments. Interpreting these measurements requires software that accounts for chemical modifications, charge states, and isotope composition ([Angel et al., 2012](#)). **Peptacular** is a Python library for representing and analyzing peptide and protein sequences using ProForma notation. It supports sequence editing, mass and mass-to-charge ratio (m/z) determination, elemental composition, isotope envelopes, enzymatic digestion, theoretical fragmentation, and physicochemical properties. Its chemical analysis APIs operate on individual peptide chains, while separate interfaces represent chimeric assignments and structured notation. Table 1 distinguishes notation support from limits on these operations.

Statement of Need

ProForma standardizes how peptide and protein sequences describe modifications and ambiguity ([LeDuc et al., 2022](#)). Version 2.1 extends this notation with additional chemical and structural annotations ([HUPO Proteomics Standards Initiative, 2026](#)). Proteomics developers still need to carry those annotations through sequence editing, digestion, and chemical analysis without silently discarding information. Peptacular targets researchers building analysis scripts, theoretical peptide libraries, and proteomics software that require consistent behavior across these operations.

The library provides scalar and batch interfaces for serialized sequences and parsed annotations. Batch results can be assigned to tabular data, including pandas DataFrames ([The pandas development team, 2025](#)). Streaming input and optional error collection support workflows in which some annotations are unresolved or unsuitable for a requested operation.

State of the Field

Existing packages address overlapping needs. **Pyteomics** ([Goloborodko et al., 2013](#)) supports ProForma parsing, mass and composition calculations, and fragment generation alongside its historical modX format ([Pyteomics developers, n.d.](#)). **Biopython** ([Cock et al., 2009](#)) provides general sequence analysis, including protein properties. The **mzcore** and related RustyMS libraries provide ProForma support, chemical representations, and theoretical fragmentation, with Python bindings for selected components ([Schulte et al., n.d.](#)). **pyOpenMS** ([Röst et al., 2014](#)) exposes a broader mass spectrometry toolkit. Current OpenMS documentation also describes ProForma parsing and conversion ([OpenMS developers, n.d.](#)).

Peptacular's contribution is a common, editable ProForma annotation model shared by sequence transformations and chemical analysis in Python. This design allows researchers to inspect and modify annotations directly while using consistent charge, modification, and ambiguity

39 handling across operations. A dedicated package keeps this interface focused on sequence
40 analysis. Optional adapters connect it to Pyteomics, psm_utils, and AlphaBase for their
41 supported representations, with checks for conversion losses. These integrations complement
42 the specialized capabilities of the surrounding ecosystem.

43 Software Design

44 Peptacular offers functional and object-oriented APIs. ProFormaAnnotation objects support
45 inspection, serialization, and chained edits. Many editing methods modify the object by
46 default and accept inplace=False to return a copy. Functional operations accept strings or
47 annotations and support sequence batches.

48 Execution can be sequential, threaded, or process-based. Automatic functional calls use
49 sequential processing below 1,000 inputs, avoiding worker startup costs for small batches.
50 Larger batches use process-based execution on conventional GIL-enabled Python builds and
51 threads on free-threaded builds. Explicit settings override this selection. Functional calls
52 create temporary pools, while iter_batch() reuses an executor across chunks within a call
53 and returns ordered results with optional diagnostics.

54 Lazy modification parsing and bounded caches reduce repeated work. Direct residue mass
55 lookups and a scalar path for ordinary precursor mass and m/z avoid unnecessary fragment
56 objects and annotation copies. A composition-based path handles isotope labels and elemental
57 adjustments. Both paths account for intrinsic modification charge and electron mass. BRAIN
58 recurrences generate aggregated isotope envelopes with a probability-weighted center mass
59 per nominal isotope peak (Dittwald & Valkenburg, 2014). Fine structure is not resolved. A
60 separate average API estimates envelopes from mass alone.

61 Shared reference data are supplied by Tacular (P. Garrett, 2026), including Unimod (Creasy &
62 Cottrell, 2004), PSI-MOD (HUPO Proteomics Standards Initiative, n.d.-a), RESID (Protein
63 Information Resource, n.d.), XLMOD (HUPO Proteomics Standards Initiative, n.d.-b), and
64 GNOme (GlyGen, n.d.). Embedded data avoid runtime ontology downloads. Chemical analyses
65 require a resolvable mass or elemental composition, depending on the operation. Unresolved
66 annotations can still be represented, while diagnostics distinguish parsing, validation, and
67 analysis failures.

68 Versioned JSON serialization preserves annotation structure and validates input against a
69 closed set of supported types. Streaming FASTA input supports plain and gzip files. An
70 optional local Model Context Protocol interface exposes the same sequence operations to
71 agent clients. The core package requires Python 3.12 or later and Tacular, with additional
72 dependencies installed only for optional integrations. Type annotations support static analysis.
73 Continuous integration runs tests, linting, type checks, and package builds across Python
74 3.12-3.14 and Linux, macOS, and Windows configurations.

75 Research impact statement

76 Peptacular has been used outside its development group. Malsagova and colleagues used it to
77 match theoretical b- and y-ions, including water and ammonia losses, when presenting peptide
78 identifications from plasma proteomics in a study of exercise intensity (Malsagova et al., 2026,
79 fig. 4). HUPO-PSI also lists Peptacular among Python implementations of ProForma (HUPO
80 Proteomics Standards Initiative, 2026). Within the authors' software ecosystem, Spxtacular
81 uses Peptacular's theoretical fragments in spectrum matching and scoring (P. T. Garrett &
82 Yates, 2026). Peptacular has additionally been used by its developers to prepare figures for a
83 proteomics textbook chapter (P. T. Garrett et al., 2025).

84 Example Usage

85 Object-oriented API

```
import peptacular as pt

# Parse a sequence into a ProFormaAnnotation
peptide: pt.ProFormaAnnotation = pt.parse("PEM[Oxidation]TIDE")

# Calculate mass and m/z
mass: float = peptide.mass() # 849.343
mz: float = peptide.mz(charge=2) # 425.679

# Chained edits modify the annotation
print(peptide.set_charge(2).set_peptide_name("Peptacular").serialize())
# (>Peptacular)PEM[Oxidation]TIDE/2
```

86 Functional API

```
import peptacular as pt

peptides = ['[Acetyl]-PEPTIDES', '<13C>ARE', 'SICK/2']

# Calculate mass and m/z for all peptides
masses: list[float] = pt.mass(peptides) # [928.4026, 388.2384, 451.2454]
mzs: list[float] = pt.mz(peptides, charge=2) # [465.2086, 195.1265, 225.6227]
```

87 Tabular workflow

```
import peptacular as pt
import pandas as pd

df = pd.DataFrame(
    {
        "seq": ["PEM[Oxidation]TIDE", "ACDEFGHIK", "AS[Phospho]TPEK"],
    }
)

df["mass"] = pt.mass(df["seq"].tolist())
```

88 Notation support and operation limits

89 Table 1: Representative ProForma 2.1 notation support and operation limits

S	Feature	Example	\$ [Level]
Y	Amino acids (+UO)	AAHCFKUOT	6.1 [1]
Y	Unimod names	PEM[Oxidation]AT	6.2.1 [1]
Y	PSI-MOD names	PEM[monohydroxylated residue]AT	6.2.1 [1]
Y	Unimod numbers	PEM[UNIMOD:35]AT	6.2.2 [1]
Y	PSI-MOD numbers	PEM[MOD:00425]AT	6.2.2 [1]
Y	Delta masses	PEM[+15.995]AT	6.2.3 [1]
Y	N-terminal modifications	[Carbamyl]-QPEPTIDE	6.3 [1]
Y	C-terminal modifications	PEPTIDEG-[Methyl]	6.3 [1]
Y	Labile modifications	{Glycan:Hex}EM[U:Oxidation]EV	6.4 [1]

S	Feature	Example	§ [Level]
Y	Multiple modifications	MPGNW[Oxidation][Carboxymethyl]PESQE	6.5 [1]
Y	Information tags	ELV[INFO:AnyString]IS	6.6 [1]
Y	Ambiguous amino acids	BZJX	7.1 [2]
Y	Prefixed delta masses	PEM[U:+15.995]AT	7.2 [2]
Y	Mass gap	PEX[+147.035]AT	7.3 [2]
Y	Formulas	PEM[Formula:0]AT, PEM[Formula:[1701]]AT	7.4 [2]
Y	Mass with interpretation	PEM[+15.995 Oxidation]AT	7.5 [2]
Y	Unknown mod position	[Oxidation]?PEMAT	7.6.1 [2]
Y	Set of positions	PEP[Oxidation#1]M[#1]AT	7.6.2 [2]
Y	Range of positions	PRT(ESFRMS)[+19.0523]ISK	7.6.3 [2]
Y	Position scores	PEP[Oxidation#1(0.95)]M[#1(0.05)]AT	7.6.4 [2]
Y	Range position scores	(PEP)[Oxidation#1(0.95)]M[#1(0.05)]AT	7.6.5 [2]
Y	Amino acid ambiguity	(?VCH)AT	7.7 [2]
Y	Modification prefixes	PEPM[U:Oxidation]AS[M:0-phospho-L-serine]	7.8 [2]
Y	Labile locations	{Phospho#g1}EMEVS[#g1]	7.9 [2]
Y	RESID modifications	EM[R:L-methionine sulfone]EM[RESID:AA0251]	8.1 [T]
Y	Names	(>Heavy chain)EVQLVESG	8.2 [T]
Y	XL-MOD modifications	EVTK[X:DSS]LEK[XLMOD:02001]SEFD	9.1 [X]
N	Cross-linkers (intrachain)	EVTK[X:DSS#XL1]LEK[#XL1]SEFD	9.2.1 [X]
N	Cross-linkers (interchain)	EVTK[X:DSS#XL1]L//EK[#XL1]SEFD	9.2.2 [X]
N	Branches	ED[MOD:00093#BRANCH]//D[#BRANCH]ATR	9.3 [X]
Y	GNO modifications	NEEYN[GNO:G59626AS]K	10.1 [G]
Y	Glycan compositions	NEEYN[Glycan:Hex5HexNAc4NeuAc1]K	10.2 [G]
Y	Mixed glycan components	N[Glycan:Hex{H2O}{+204.068}]K	10.2 [G]
Y	Charged formulas	SEQUEN[Formula:Zn1:z+2]CE	11.1 [3]
P	Controlling placement	PTI(MERMERME)[+32 Position:E]PTIDE	11.2 [3]
Y	Global isotope	<13C>PEPTIDE	11.3.1 [3]
Y	Fixed modifications	<[Oxidation]@M>ATPEMILTCMGLK	11.3.2 [3]
Y	Chimeric spectra	NEEYN+SEQUEN	11.4 [3]
Y	Charges	SEQUEN/2, SEQUEN/[Na:z+1,H:z+1]	11.5 [3]
Y	Ion notation	SEQUEN-[b-type-ion]	11.6 [3]

Table 1 summarizes notation handling against the finalized ProForma 2.1 specification (HUPO Proteomics Standards Initiative, 2026). In column S, Y indicates supported notation, P indicates a preserved annotation whose placement constraints are not applied, and N indicates linked-chain notation that can be represented but is not supported by the analysis operations. Levels 1, 2, and 3 follow the specification, with top-down (T), cross-linking (X), and glycan (G) extensions. Notation support does not imply that every operation is available for every represented sequence.

Mass-ambiguous residues B and Z cannot produce a unique mass. Delta-mass tags and bare-mass glycan components lack elemental compositions. Unknown localization and ambiguous intervals restrict fragmentation. Labile modifications contribute to precursor mass but are omitted from fragment ions. Chimeric assignments are handled component-wise through `parse_chimeric()`. The structured model and JSON format can represent linked notation, but the analysis APIs do not operate on cross-linked or branched structures. The documentation provides operation-specific limits and examples.

AI usage disclosure

Opus 5 and Fable 5.1 through Claude Code, and Sol and Astra through OpenAI Codex, assisted with code generation, refactoring, tests, debugging, documentation, and manuscript revision. Verification included automated tests, static analysis, execution of manuscript examples, and checks against primary references. The human authors reviewed, edited, and validated all AI-assisted work and made the core design decisions.

Availability

Peptacular is distributed through PyPI (<https://pypi.org/project/peptacular/>) and available as open-source software on GitHub (<https://github.com/tacular-omics/peptacular>). Documentation is accessible at <https://peptacular.readthedocs.io>. The software is released under the MIT license.

Acknowledgements

This work was supported by the National Institutes of Health under grants R01 AG077046 (Analysis of protein interactions in neurodegenerative disease), R01 MH132570 (Brain-wide mapping of neuronal inhibition by novel inverse activity markers), R01 MH100175 (Proteogenetics of Autism Spectrum Disorders), R01 HL165168 (The CFTR Interactome), and U01 AG088679 (Understanding Gene-Environment Interactions in Brain Aging and Alzheimer's Disease (AD) and AD-Related Dementias (ADRD)).

The funders provided financial support only.

Competing interests

The authors report no competing interests.

References

- Angel, T. E., Aryal, U. K., Hengel, S. M., Baker, E. S., Kelly, R. T., Robinson, E. W., & Smith, R. D. (2012). Mass spectrometry-based proteomics: existing capabilities and future directions. *Chemical Society Reviews*, 41(10), 3912–3928. <https://doi.org/10.1039/c2cs15331a>
- Cock, P. J. A., Antao, T., Chang, J. T., Chapman, B. A., Cox, C. J., Dalke, A., Friedberg, I., Hamelryck, T., Kauff, F., Wilczynski, B., & De Hoon, M. J. L. (2009). Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422–1423. <https://doi.org/10.1093/bioinformatics/btp163>
- Creasy, D. M., & Cottrell, J. S. (2004). Unimod: Protein modifications for mass spectrometry. *PROTEOMICS*, 4(6), 1534–1536. <https://doi.org/10.1002/pmic.200300744>
- Dittwald, P., & Valkenborg, D. (2014). BRAIN 2.0: Time and Memory Complexity Improvements in the Algorithm for Calculating the Isotope Distribution. *Journal of the American Society for Mass Spectrometry*, 25(4), 588–594. <https://doi.org/10.1007/s13361-013-0796-5>
- Garrett, P. (2026). *tacular-omics/tacular: v1.0.1 - Zenodo + Docs* (Version v1.0.1). Zenodo. <https://doi.org/10.5281/zenodo.18475557>
- Garrett, P. T., Turner, N. P., Nakorchevsky, A., Pankow, S., & Yates, J. R. (2025). Mass spectrometry-based proteomics. In *Reference module in life sciences*. Elsevier. <https://doi.org/10.1016/b978-0-323-99507-8.00013-5>

- 144 Garrett, P. T., & Yates, J. R., III. (2026). *Spxtacular*. [https://github.com/tacular-omics/](https://github.com/tacular-omics/spxtacular)
145 [spxtacular](https://github.com/tacular-omics/spxtacular)
- 146 GlyGen. (n.d.). *GNOME: Glycan Naming and Subsumption Ontology*. Retrieved September
147 13, 2026, from <https://github.com/glygen-glycan-data/GNOME>
- 148 Goloborodko, A. A., Levitsky, L. I., Ivanov, M. V., & Gorshkov, M. V. (2013). Pyteomics:
149 A Python framework for exploratory data analysis and rapid software prototyping in
150 proteomics. *Journal of the American Society for Mass Spectrometry*, 24(2), 301–304.
151 <https://doi.org/10.1007/s13361-012-0516-6>
- 152 HUPO Proteomics Standards Initiative. (n.d.-a). *PSI-MOD ontology for modified and*
153 *unmodified amino acid residues*. Retrieved September 13, 2026, from [https://github.com/](https://github.com/HUPO-PSI/psi-mod-CV)
154 [HUPO-PSI/psi-mod-CV](https://github.com/HUPO-PSI/psi-mod-CV)
- 155 HUPO Proteomics Standards Initiative. (n.d.-b). *XLMOD ontology for chemical cross-linkers*.
156 Retrieved September 13, 2026, from <https://github.com/HUPO-PSI/xlmod-CV>
- 157 HUPO Proteomics Standards Initiative. (2026). *ProForma 2.1: Proteoform and Peptidoform*
158 *Notation*. <https://www.psidev.info/proforma>
- 159 LeDuc, R. D., Deutsch, E. W., Binz, P.-A., Fellers, R. T., Cesnik, A. J., Klein, J. A., Van
160 Den Bossche, T., Gabriels, R., Yalavarthi, A., Perez-Riverol, Y., Carver, J., Bittremieux,
161 W., Kawano, S., Pullman, B., Bandeira, N., Kelleher, N. L., Thomas, P. M., & Vizcaíno,
162 J. A. (2022). Proteomics Standards Initiative's ProForma 2.0: Unifying the Encoding
163 of Proteoforms and Peptidoforms. *Journal of Proteome Research*, 21(4), 1189–1195.
164 <https://doi.org/10.1021/acs.jproteome.1c00771>
- 165 Malsagova, K. A., Butkova, T. V., Nikolsky, K. S., Petrovskiy, D. V., Kopylov, A. T., Nakhod,
166 V. I., Kulikova, L. I., Rudnev, V. R., Yurku, K. A., Balakin, E. I., Bukreeva, A. S., Izotov,
167 A. A., Pustovoyt, V. I., & Kaysheva, A. L. (2026). Changes in the proteomic profile
168 of athletes' plasma associated with exercise intensity. *Scientific Reports*, 16(1), 14205.
169 <https://doi.org/10.1038/s41598-026-44729-5>
- 170 OpenMS developers. (n.d.). *OpenMS ProForma class reference*. Retrieved September 13,
171 2026, from https://openms.de/current_doxygen/html/classOpenMS_1_1ProForma.html
- 172 Protein Information Resource. (n.d.). *RESID Database*. Retrieved September 13, 2026, from
173 <https://proteininformationresource.org/resid/>
- 174 Pyteomics developers. (n.d.). *Pyteomics: Peptide sequence formats*. Retrieved September 13,
175 2026, from <https://pyteomics.readthedocs.io/en/latest/sequences.html>
- 176 Röst, H. L., Schmitt, U., Aebersold, R., & Malmström, L. (2014). pyOpenMS: A Python-based
177 interface to the OpenMS mass-spectrometry algorithm library. *PROTEOMICS*, 14(1),
178 74–77. <https://doi.org/10.1002/pmic.201300246>
- 179 Schulte, D., Gabriels, R., & Heerdink, A. (n.d.). *mzcore*. Retrieved September 13, 2026, from
180 <https://github.com/rusteomics/mzcore>
- 181 The pandas development team. (2025). *pandas-dev/pandas: Pandas (Version 3.0.0rc1)*.
182 Zenodo. <https://doi.org/10.5281/zenodo.17992932>